

Guía de entregables

Este directorio contiene los entregables listos para el despliegue del sistema en los servidores.

Estructura de Carpetas en este Directorio

- `README.md` : Este archivo con las instrucciones.
- `estructura_db.sql` : Script SQL nativo con la estructura de la base de datos vacía (PostgreSQL).
- `frontend/` : Carpeta con los archivos estáticos de la web compilados con la ruta relativa `/api`.
- `backend/` : Código fuente de la API Node.js/Express listo para producción.

1. Despliegue del Backend (API)

El backend es un servicio Node.js que utiliza Prisma ORM para estructurar y comunicarse con la base de datos PostgreSQL.

Preparación de la Base de Datos

- Es importante contar con la DB creada en el servidor de PostgreSQL antes de continuar.

Instalación y Configuración

1. Copiar el contenido de la carpeta `backend/` al directorio de destino en el servidor (ejemplo: `/opt/scyt-api`).
2. Crear el archivo `.env` en la raíz de esa carpeta basándose en el archivo de ejemplo `.env.example` que se puede encontrar en `backend/.env.example`:

```
PORT=8000
DATABASE_URL=postgresql://usuario:contraseña@servidor_db:5432/nombre_db?schema=public
JWT_SECRET=una_clave_secreta_segura_para_jwt
INITIAL_ADMIN_USER=SCyT-Admin
INITIAL_ADMIN_PASSWORD=una_contraseña_segura_para_el_admin
```

Inicialización de la Base de Datos y Dependencias

Desde la carpeta del backend en el servidor, ejecutar los siguientes comandos para instalar las dependencias de producción, generar el cliente ORM y crear la estructura de tablas de manera automática:

```
# 1. Instalar dependencias excluyendo las de desarrollo
```

```
npm ci --omit=dev
```

```
# 2. Generar el cliente interno de Prisma
```

```
npx prisma generate
```

```
# 3. Sincronizar y crear las tablas en la base de datos
```

```
npx prisma db push
```

(Nota: Al levantar la API por primera vez, el servidor sembrará automáticamente los roles, regionales, tipos de proyecto y creará el usuario administrador inicial utilizando las credenciales configuradas en el archivo `.env`).

Iniciar el Servicio

Ejecutar el servicio en segundo plano usando un gestor de procesos (ejemplo con PM2):

```
pm2 start src/app.js --name "scyt-api"
```

Nota para Entornos con Accesos Restringidos

Si el servidor de la aplicación no cuenta con permisos para crear o modificar tablas de forma directa en el servidor de base de datos, se provee el script SQL nativo `estructura_db.sql` en la raíz de este directorio para importar la estructura manualmente (saltandonos el paso de `npx prisma db push`):

```
psql -U usuario -d nombre_db -f estructura_db.sql
```

2. Despliegue del Frontend (Web)

El frontend se entrega pre-compilado en la carpeta `frontend/` y está configurado para realizar peticiones de forma relativa a la ruta `/api` de la web.

1. Copia todo el contenido de la carpeta `frontend/` al directorio de archivos estáticos de tu servidor web.
2. Es importante configura el servidor web para realizar dos tareas fundamentales:
 - Redirigir todas las peticiones entrantes a `/api/` hacia la dirección interna del backend (ej. `http://localhost:8000/api/`).
 - Redirigir cualquier otra ruta que no corresponda a un archivo físico hacia `index.html` para permitir el enrutamiento de React (React Router).

Ejemplo de Configuración Nginx con estas dos tareas:

```
server {  
    listen 80;  
    server_name tu-dominio-facultad.edu.ar;  
  
    root /var/www/scyt-web;  
    index index.html;  
  
    # Proxy inverso para la API  
    location /api/ {  
        proxy_pass http://localhost:8000/api/;  
        proxy_http_version 1.1;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
    }  
  
    # SPA Fallback  
    location / {  
        try_files $uri $uri/ /index.html;  
    }  
}
```

(Nota: En la carpeta `scyt-web/` del código original del repositorio se puede encontrar plantillas completas de `nginx.conf` y `apache.conf` de referencia, utilizadas para el despliegue en estos servidores dentro de docker).